A Comparative Study Of Lightweight Face Detection Models For Real-Time Mobile Applications

S. Balaji[1*] and Priyanka CP[2*]

[1*]Department of Computer Science, Pondicherry University, Puducherry, 605014, India.
[2*]Department of Computer Science, Guest Faculty JNRM, Sri Vijay Puram, 744101, India.

Abstract

Real-time facial detection on mobile and edge devices is crucial for applications such as augmented reality, security, and mobile technologies. However, traditional facial detection models often demand significant computational resources, mak- ing them unsuitable for resource-constrained environments. This paper evaluates several lightweight facial detection models, including BlazeFace, MediaPipe Face Detection, UltraFace, tiny YOLO, MobileNetV2, single shot multibox detection, EfficientDet, OpenCV Haar cascade classifier, local binary pattern face detector, and Fast R-CNN with reduced layers. The study assesses these models on the basis of speed, accuracy, and resource efficiency in mobile and edge environments. The experimental results indicate that while these models provide competitive accuracy, their performance varies under conditions such as occlusion, lighting, and device capabilities. The findings offer practical recommendations for deploy- ing efficient face detection solutions in real-time mobile applications, balancing computational efficiency with detection accuracy

Keywords: Lightweight face detection, Mobile applications, Real-time applications, Edge computing, MediaPipe, Tiny YOLO

1 Introduction

Facial detection is a critical technology that underpins numerous applications in fields such as augmented reality (AR), mobile security, and human–computer interaction. The growing adoption of mobile devices and edge computing platforms has increased the demand for real-time face detection systems that can operate efficiently within the constraints of limited computational resources. [1] Traditional facial detection models, particularly those based on deep learning techniques, often require significant processing power, memory, and storage, making them unsuitable for deployment in resource-constrained environments such as smartphones, tablets, or IoT devices. In response to these challenges, a variety of lightweight facial detection models have been developed to provide faster and more resource-efficient solutions while maintaining high detection accuracy. Models such as BlazeFace, MediaPipe Face Detection, Ultra- Face, Tiny YOLO, and MobileNetV2 offer an attractive balance of speed and accuracy and are optimized to run efficiently on devices with limited hardware capabilities. These models employ various optimization techniques, such as model quantization, network pruning, and the use of simpler architectures, to minimize computational complexity without compromising performance. [2] However, the performance of these lightweight models can vary significantly depending on factors such as lighting con- ditions, facial occlusion, and the processing power of the device on which they are running. This paper evaluates a range of lightweight facial detection models, including state-of-the-art deep learning approaches such as single shot multibox detector (SSD), EfficientDet, and Fast R-CNN with reduced layers, alongside traditional computer vision techniques such as OpenCV Haar cascade classifier and local binary pattern (LBP) facial detection. We assess these models on the basis of three key criteria: detec- tion speed (inference time), accuracy (detection precision), and resource efficiency

(CPU/GPU usage and power consumption). These metrics are evaluated in the con- text of real-time facial detection on mobile and edge devices, where the trade-offs between accuracy and resource consumption are of paramount importance. Through extensive experimentation, this study provides insights into the strengths and weak- nesses of each model under real-world conditions. The results will help developers and researchers identify the most suitable models for specific applications, whether for mobile security, facial recognition, or other AR-based systems. The findings also provide practical recommendations on how to optimize facial detection models for real-time performance while maintaining a balance between computational efficiency and detection accuracy.

2 Related Works

Facial detection has been a crucial task in computer vision, with a wide range of approaches and models developed to address different challenges, such as accuracy, speed, and resource consumption. Owing to their simplicity and efficiency, traditional methods such as the Haar cascade classifier, introduced by Viola and Jones, have been widely used for real-time facial detection. [3] These methods use Haar-like features and a cascade of classifiers to detect faces, but they are less effective in complex environments.

2

with varying lighting or occlusions. Another traditional approach, the histogram of oriented gradients (HOG), captures gradient information to detect faces. Nevertheless, it requires significant computational resources and may not perform well in real-time applications on mobile devices. With the advent of deep learning, methods such as mul- titask cascaded convolutional networks (MTCNNs) and You Only Look Once (YOLO) have emerged, offering more accurate and robust solutions for facial detection. The MTCNN performs facial detection and alignment in a multistage process, combining bounding box regression with landmark localization, making it suitable for handling faces in different poses and lighting conditions. YOLO, on the other hand, has revolu- tionized real-time object detection, including facial detection, by framing the task as a single regression problem. YOLO models are capable of detecting multiple objects in real time, including faces, and have been widely adopted for applications that require high speed and accuracy. As facial detection has become increasingly impor- tant in mobile and embedded applications, the need for lightweight models that can run efficiently on resource-constrained devices has increased. Models such as Blaze- Face, developed by Google, have been designed specifically for this purpose. BlazeFace uses a lightweight architecture optimized for mobile devices, balancing accuracy and computational efficiency. Similarly, MediaPipe, also developed by Google, provides a framework that integrates various facial detection models, including BlazeFace, into a pipeline optimized for real-time performance on mobile platforms. MediaPipe has found applications in augmented reality (AR), where real-time facial tracking is essen- tial. [5] Another lightweight model, UltraFace, has been designed to run efficiently on edge devices while providing high detection accuracy, even in challenging scenar- ios such as low-resolution inputs or low-light environments. Recent developments also include methods that focus on improving the efficiency of existing architectures. For example, tiny YOLO, a smaller version of the YOLO framework, has been developed to perform real-time facial detection with reduced computational overhead, making it suitable for mobile and embedded devices. MobileNetV2, an efficient deep learn- ing model designed for mobile applications, has been integrated with facial detection tasks, offering a balance between accuracy and computational cost. Other models, such as the single shot multibox detector (SSD) and EfficientDet, provide efficient object detection frameworks that can be adapted for facial detection. [6] SSD is known for its ability to detect multiple objects in a single pass, whereas EfficientDet, an extension of the EfficientNet architecture, uses a compound scaling method to improve both accu- racy and efficiency. While these deep learning-based methods outperform traditional approaches in terms of accuracy, they often have increased computational and mem- ory requirements, which poses challenges for real-time deployment on mobile devices. As such, much of the recent facial detection research has

focused on optimizing these models to meet the constraints of mobile and embedded platforms. For example, the local binary patterns (LBP) face detector offers a traditional yet lightweight approach that is ideal for low-resource environments, whereas Fast R-CNN, with reduced layers, has been used tostreamline the detection process for faster performance. [7] However, achieving a balance between high accuracy, fast inference speed, and minimal resource consumption remains a significant challenge in real-time mobile face detection. Despite these advancements, face detection on mobile devices still faces challenges related to varying lighting conditions, occlusions, and the need for high-speed inference with minimal power consumption. Models such as BlazeFace and UltraFace are designed to address these challenges because they are lightweight and optimized for edge com- puting. However, there is still much room for improvement, particularly in terms of handling extreme conditions such as poor lighting, large variations in face orienta- tions, and real-time performance on low-end mobile devices. [8] Future research will likely continue to focus on optimizing existing models, creating novel architectures, and developing more efficient algorithms to make facial detection a reliable tool for mobile applications in a wide range of real-world scenarios.

3 Methodology

3.1 Model Selection

In this section, we evaluate various state-of-the-art lightweight face detection mod- els to determine the most efficient solution for real-time mobile applications. The selected models are designed to operate efficiently on resource-constrained devices while maintaining high detection accuracy. The following models are evaluated:

3.1.1 BlazeFace

BlazeFace is a lightweight, fast face-detection model developed by Google for real-time facial detection on mobile devices. The model is based on a MobileNetV1 backbone and is optimized for speed, targeting an inference time of approximately 25 ms on mobile GPUs. BlazeFace employs a multistage detection process, which combines the benefits of a shallow architecture with high detection accuracy. The key advantage of BlazeFace lies in its efficient use of convolutional layers to detect faces at multiple scales. The model uses the following form for bounding box prediction:

$$\hat{y} = \sigma(W \cdot x + b) \quad (1)$$

where x is the input feature map, W represents the weights, b the biases, and σ is the sigmoid activation function applied element-wise to predict the bounding box coordinates.

3.1.2 MediaPipe Face Detection

MediaPipe Face Detection, developed by Google, is a cross-platform framework for building pipelines that process audio, video, and sensor data. It provides a fast and accurate facial detection model that is optimized for mobile platforms. [9] MediaPipe uses a single-shot face detector based on a lightweight neural network architecture combined with an efficient postprocessing pipeline. The network architecture follows a similar detection framework as BlazeFace but with enhanced accuracy due to the use of a deeper feature extractor. The bounding box prediction in MediaPipe is given by:

$$\hat{y} = \sigma(W \cdot \text{Conv2D}(x) + b) \quad (2)$$

where Conv2D is a 2D convolutional operation, and W and b are the learned parameters.

3.1.3 UltraFace

Ultraface is a real-time facial detection model that balances accuracy and efficiency. It leverages the single shot multibox detector (SSD) architecture combined with an efficient

lightweight backbone, typically MobileNetV2, to process images with mini- mal latency. UltraFace performs exceptionally well on resource-limited devices such as smartphones while maintaining state-of-the-art accuracy. [10] The face detection out- put from UltraFace is obtained by predicting the center of the bounding box and its dimensions via:

$$y = \sigma(W \cdot \text{MobileNetV2}(x) + b) \quad (3)$$

where MobileNetV2(x) represents the output of the MobileNetV2 feature extractor applied to input x, and σ is the sigmoid activation function.

3.1.4 Tiny YOLO

Tiny YOLO is a reduced version of the original YOLO (you only look once) facial detection model, which is designed to operate at high speeds while sacrificing some detection accuracy for improved performance on mobile devices. Compared with its full counterpart, tiny YOLO reduces the number of layers and parameters, making it more efficient. The output of tiny YOLO is formulated as:

$$\hat{y} = \text{sigmoid}(W \cdot \text{Conv2D}(x) + b) \quad (4)$$

where the Conv2D operation is used to detect multiple faces at once, and the output is passed through a sigmoid function for bounding box regression.

3.1.5 MobileNetV2

MobileNetV2 is a lightweight convolutional neural network designed for mobile and embedded vision applications. It uses depthwise separable convolutions to reduce com- putational complexity. Although MobileNetV2 is not a facial detection model by itself, it is often used as a backbone for object and face detection tasks. The output of MobileNetV2 when it is applied to facial detection tasks is similar to that of the previous models:

$$y = \sigma(W \cdot \text{MobileNetV2}(x) + b) \quad (5)$$

where x is the input image, and W and b represent the learned parameters for detecting faces.

3.1.6 Single Shot Multibox Detector (SSD)

The SSD model is a popular facial detection approach that detects faces at multiple scales within a single shot. SSD uses a convolutional feature map to predict bounding boxes and corresponding class labels directly, eliminating the need for region proposal networks. The prediction formula for SSD is as follows:

$y = \text{softmax}(W \cdot \text{Conv2D}(x) + b)$ (6) where the softmax function is used to calculate the confidence scores for the predicted bounding boxes.

3.1.7 EfficientDet

EfficientDet is a family of models that uses a compound scaling method to optimize the architecture for both accuracy and efficiency. [18] The model scales the width, depth, and resolution of the network according to a fixed ratio, resulting in a compact yet powerful model for real-time applications. The detection output from EfficientDet is derived as follows:

$$y = \sigma(W \cdot \text{EfficientNet}(x) + b) \quad (7) \text{ where}$$

EfficientNet(x) is the backbone network optimized for mobile platforms.

3.1.8 OpenCV Haar Cascade Classifier

The Haar cascade classifier is a traditional machine learning approach for facial detec- tion that is based on Haar features. [17] This method is often used in real-time applications because of its speed and simplicity. The model is based on the detection of rectangular features at different scales. The classifier works by applying a series of weak classifiers to the input image, where the

overall decision is made via:

$$f(x) = \sum_{i=1}^{n} \alpha_i h_i(x) \quad (8)$$

where $h_i(x)$ are the weak classifiers (Haar features), α_i are their respective weights, and $f(x)$ is the final classification score.

3.1.9 Local Binary Patterns Face Detector (LBP)

Local binary patterns (LBPs) are another traditional facial detection method that is particularly well suited for detecting faces in controlled environments with good lighting. [16] It relies on texture patterns of the image to differentiate between facial regions and nonfacial regions. The LBP operation computes the binary pattern for each pixel and assigns a label based on the surrounding neighborhood, with the classification function:

$$y = \text{LBP}(x) \quad (9)$$

where LBP(x) generates a texture descriptor that is used for face classification.

3.1.10 Fast R-CNN (with reduced layers)

Fast R-CNN is a region-based convolutional neural network for object detection, including facial detection. It improves upon the original R-CNN by using a single for- ward pass of the network and applying region-of-interest (RoI) pooling. [12] In this reduced version, the network layers are optimized to balance speed and accuracy. The Fast R-CNN prediction formula for facial detection is as follows:

The Fast R-CNN prediction formula for face detection is:

$$y = \text{softmax}(W \cdot \text{RoIPool}(\text{Conv}(x)) + b) \quad (10)$$

where RoIPool is used to extract region-based features, followed by the softmax activation to predict the bounding box classes.

4 Dataset Selection

In this section, we present the datasets selected for evaluating the performance of the face detection models discussed in Section 3. These datasets provide a diverse range of images and video streams to assess model accuracy, robustness, and real-world applicability. The selected datasets include both image-based and video-based data, capturing faces in various conditions and environments.

4.1 WIDER FACE Dataset

The WIDER FACE dataset is one of the most widely used datasets for facial detec- tion. It is designed for evaluating facial detection algorithms across a wide range of challenging conditions, including variations in scale, pose, occlusion, and illumina- tion. The WIDER FACE dataset contains 32,203 images and over 400,000 labeled faces, which are divided into three subsets: training, validation, and testing. [15] The dataset is collected from the internet and contains images from various sources, such as movies, social media, and personal photos. The dataset provides several challenges for face detection models because of its diverse scenarios. For example, faces may be seen from various angles, partially occluded, or appear in low-resolution images. This makes WIDER FACE a highly relevant dataset for evaluating the robustness of facial detection models.

The WIDER FACE dataset is particularly useful for the following:

- Testing face detection under varying conditions such as illumination and occlusion.
- Evaluating the performance of models on highly diverse and real-world image sets.

4.2 LFW Dataset

The labeled faces in the Wild (LFW) dataset is another popular dataset for evaluating facial

recognition and detection systems. LFW contains 13,000 labeled images of faces from 5,749 different individuals collected from the internetInternet. [14] These images are captured in uncontrolled settings, with variations in pose, lighting, and expression, making the dataset particularly challenging for facial detection models.

LFW consists of 10 different subsets:

- Training set: 7,000 images (including positive and negative samples).
- Testing set: 6,000 images for model evaluation.
- All images are labeled with the name of the individual they represent.

For face detection, LFW is useful because:

- It provides a large-scale collection of faces in real-world conditions.
- It is widely used for benchmarking models on face verification and recognition tasks.

The LFW dataset is ideal for evaluating the detection of faces in real-world, uncon- strained environments where factors like variation in appearance, lighting, and facial expressions are present.

4.3 Real-World Video Stream Collection

In addition to image-based datasets, we also utilize a collection of real-world video streams to evaluate the performance of the facial detection models in dynamic, real- time scenarios. Video streams are captured in diverse environments, such as crowded public spaces, indoors with varying lighting conditions, and outdoor settings with dif- ferent weather conditions. This dataset is not prelabeled and requires the application of facial detection algorithms to identify and annotate faces in video frames. The collection of video streams is captured via a variety of camera systems, including smart- phones, security cameras, and consumer-grade video cameras. The video resolution and frame rates vary, adding additional complexity to the facial detection task

Real-world video stream collections are important because:

- They provide dynamic and evolving data for evaluating model performance in real time.
- They simulate real-world use cases where faces must be detected across frames of a video, under varying conditions.
- These datasets help evaluate the temporal consistency and tracking of faces across frames in a video sequence.

The real-world video stream collection will be used to test the models' ability to detect faces in complex, real-time video scenarios, where performance consistency across frames is critical.

5 Experimental Setup

In this section, we outline the experimental setup used to evaluate the face detection models discussed in Section 3. This includes the hardware devices on which the models were tested and the benchmarking framework used for performance evaluation.

5.1 Mobile Devices Used

14(

The performance of the facial detection models was evaluated on a variety of mobile and edge devices to ensure that the models were optimized for real-world applications.

[13] These devices are selected on the basis of their widespread use and their varying computational capabilities, which allows for a comprehensive evaluation of the mod- els under different hardware constraints. The devices used in this study include the following:

- Smartphones:
  - Apple iPhone 14 Pro (A16 Bionic chip, 6 GB RAM)
  - Google Pixel 7 Pro (Google Tensor chip, 12 GB RAM)
  - Samsung Galaxy S22 Ultra (Exynos 2200 chip, 12 GB RAM)

These devices were chosen to represent a wide range of performance levels, from high-end

smartphones with advanced processors to lower-power edge devices suit- able for real-time edge computing applications.[19] The devices were used to evaluate how well the models perform in both mobile and embedded system contexts, where computational resources are often limited.

5.2 Benchmarking Framework

To evaluate the performance of the facial detection models on the selected devices, we employed a standardized benchmarking framework. The framework focuses on key metrics relevant to real-time facial detection applications, including inference time, accuracy, and computational efficiency. The following components were considered in the benchmarking process:

- Inference Time: The time taken by the model to process a single frame of an image or a video stream. This is critical for real-time applications and affects the usability of the model in edge devices.
- Accuracy: We report the mean average precision (mAP) and precision-recall curves to evaluate the model's detection performance. The accuracy is calculated on both the WIDER FACE and LFW datasets, as discussed in Section 4.
- Memory Usage: The amount of memory (RAM) utilized by the model dur- ing inference. This metric is important for evaluating the model's suitability for resource-constrained devices.
- Power Consumption: The average power consumption of the devices while running the face detection models. This is especially important for mobile and edge devices, which often operate on battery power.
- Throughput: The number of frames per second (FPS) processed by the model, which indicates its ability to handle video streams in real time.
- Energy Efficiency: The energy consumed per frame processed, which is crucial for mobile and edge devices that need to maximize battery life.

The benchmarking tests were performed under various environmental conditions, such as different lighting, occlusion levels, and background noise, to ensure that the models can handle a range of real-world scenarios.

Software Setup: The experiments were conducted using the following software tools and frameworks:

- TensorFlow Lite for model inference on mobile devices.
- PyTorch for benchmarking on edge devices with GPU acceleration.
- OpenCV for video streaming and image pre-processing tasks.
- Custom benchmarking scripts for measuring inference time, accuracy, memory usage, and power consumption.

By using these benchmarking metrics and hardware devices, we ensure that the models are thoroughly evaluated for practical deployment in real-world face detection applications, both on mobile and edge devices.

6 Evaluation Metrics

In this section, we define the evaluation metrics used to assess the performance of the facial detection models. The metrics provide insights into the model's accuracy, speed, and resource usage, which are critical for real-time deployment on mobile devices.

6.1 Detection Accuracy

Detection accuracy is one of the most important performance metrics for facial detec- tion models. We measure accuracy via the **mean average precision (mAP)**, which is widely used in object detection tasks.

6.1.1 Mean Average Precision (mAP)

The mean average precision (mAP) is calculated as the mean of the average precision (AP) scores

for each class. It is computed by first calculating the precision and recall for each detection and then using these values to compute the average precision.
The average precision for each class is defined as:

$$AP = \int_0^1 P(r)\, dr \quad (11)$$

where $P(r)$ is the precision at recall r, and it is computed as:

$$P(r) = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \quad (12)$$

The precision-recall curve is computed for each detected object, and the area under the curve (AUC) gives the AP for that class. The mAP is then computed by averaging the AP scores across all classes:

$$mAP = \frac{1}{C} \sum_{i=1}^{C} AP_i \quad (13)$$

where C is the number of classes in the dataset.

6.2 Inference Speed (FPS)

Inference speed, measured in frames per second (FPS), refers to how quickly the model processes a single frame of input data. This is a crucial metric for evaluating the real-time performance of face detection models, especially for mobile devices.
To calculate the FPS, we use the following formula:

$$FPS = \frac{\text{Total number of frames processed}}{\text{Total time taken for processing}} \quad (14)$$

For instance, if a model processes 1000 frames in 50 seconds, the FPS would be:

$$FPS = \frac{1000}{50} = 20 \text{ frames per second} \quad (15)$$

This metric helps evaluate how well the model performs on mobile devices, considering their computational limitations.

6.3 Resource Usage (CPU/GPU, Memory, Power)

Resource usage is another important evaluation metric, as it reflects how much of the mobile device's resources (CPU/GPU, memory, and power) the model consumes during inference. These metrics are critical for assessing the efficiency of the model, especially for real-time applications on mobile devices.

6.3.1 CPU/GPU Usage

CPU/GPU usage is measured using profiling tools, which report the percentage of the CPU or GPU utilized during model inference. This can be calculated as:

$$\text{CPU/GPU Usage} = \frac{\text{Time spent on computation}}{\text{Total time}} \times 100 \quad (16)$$

For example, if the model utilizes 30% of the GPU for a given task, then the GPU usage is 30%.

6.3.2 Memory Usage

Memory usage refers to the amount of RAM used by the model during inference. It can be measured in MB or GB, depending on the system. Memory usage is critical for evaluating the model's ability to run on resource-constrained mobile devices.

$$\text{Memory Usage} = \text{Peak memory consumption during inference} \quad (17)$$

For instance, if the model requires 500 MB of RAM during processing, this value is reported as the memory usage.

6.3.3 Power Consumption

Power consumption is the amount of power drawn by the mobile device during model inference, typically measured in watts (W). This is important for mobile devices, as excessive power usage can drain the battery quickly.
To calculate power consumption, we can use:

$$\text{Power Consumption} = \text{Voltage} \times \text{Current} \quad (18)$$

For example, if the mobile device operates at 5V and draws 2A of current during inference, the power consumption would be:

$$\text{Power Consumption} = 5\ \text{V} \times 2\ \text{A} = 10\ \text{W} \quad (19)$$

This is essential for assessing the energy efficiency of the model on mobile devices.

7 Performance Testing

In this section, we describe the methodology for testing the performance of face detection models on mobile devices under both controlled and real-world conditions.

7.1 Controlled Environment Testing

In controlled environment testing, the models are evaluated under predefined, ideal conditions. These conditions include uniform lighting, clear visibility of faces, and minimal occlusion. The primary objective is to assess the models' detection accuracy and inference speed in a controlled setting, where external variables are minimized.

7.1.1 Test Setup

For controlled environment testing, the following conditions were set:

- Lighting: Uniform lighting with no significant shadows.
- Face Placement: The faces are placed at varying distances from the camera (e.g., 1m, 2m, and 3m).
- Occlusion: Faces with minimal occlusion (e.g., no hands or objects blocking the face).

7.1.2 Procedure

The following steps were followed during the testing: 1. A series of images or video streams containing faces was captured under controlled conditions. 2. Each model was applied to the dataset, and the number of frames processed per second (FPS) was recorded. 3. The detection accuracy (mAP) and inference time for each model were calculated. 4. Resource usage, including CPU/GPU usage, memory consumption, and power consumption, was monitored and recorded.

7.2 Real-World Testing

Real-world testing evaluates the model's performance under more challenging con- ditions, such

as dynamic lighting, varying facial orientations, partial occlusions, and diverse backgrounds. The aim is to assess how well the model can generalize to diverse real-world scenarios on mobile devices.

7.2.1 Test Setup

For real-world testing, the models were tested in environments that more closely resemble real-world face detection tasks, including:

- Lighting: Varied lighting conditions, including bright sunlight, low-light conditions, and mixed lighting.
- Facial Orientation: Faces in different orientations (e.g., frontal, profile, tilted).
- Occlusion: Faces with partial occlusions (e.g., hats, glasses, hands).
- Background Variability: Crowded or complex backgrounds.

7.2.2 Procedure

1. Video streams were captured from real-world scenarios, including street scenes, indoor events, and public spaces. 2. The models processed the video streams, and FPS, detection accuracy, and resource usage were recorded. 3. The models' ability to handle challenging conditions such as occlusion and low light was tested, and performance metrics were reported.

8 Face Detection Algorithms

Algorithm 1 BlazeFace Face Detection Algorithm

Require: Input image I of size $h \times w$
Ensure: Detected faces in an image
1: Preprocess image I for input to BlazeFace
2: Apply lightweight CNN with a small stride
3: Extract feature maps from the network layers
4: Detect bounding boxes using default anchor boxes
5: Apply non-maximum suppression (NMS) to remove redundant detections
6: Return final bounding boxes with confidence scores

Algorithm 2 MediaPipe Face Detection Algorithm

Require: Input image I of size $h \times w$ **Ensure:** Detected faces in an image 1: Convert image I to RGB
2: Normalize the image for processing
3: Pass image through a deep neural network (DNN) based on MobileNetV2
4: Detect bounding boxes and key facial landmarks
5: Apply thresholding and NMS for accurate bounding box selection
6: Return detected faces and landmarks

Algorithm 3 UltraFace Face Detection Algorithm
Require: Input image *I* of size $h \times w$
Ensure: Detected faces in an image
1: Preprocess image *I* by resizing and normalizing
2: Pass the preprocessed image through the UltraFace model 3: Use multiscale feature maps to detect faces at various scales 4: Apply regression-based bounding box prediction
5: Use a softmax classifier for face detection confidence
6: Apply NMS to filter overlapping boxes
7: Return the final detected faces

Algorithm 4 Tiny YOLO Face Detection Algorithm
Require: Input image *I* of size $h \times w$
Ensure: Detected faces in an image
1: Preprocess image by resizing to 416×416
2: Pass image through the Tiny YOLO network
3: Extract bounding box predictions from the output grid
4: Apply sigmoid activation to the class predictions
5: Use NMS to suppress low-confidence detections and overlapping boxes
6: Return final bounding boxes and class labels

Algorithm 5 MobileNetV2 Face Detection Algorithm
Require: Input image *I* of size $h \times w$
Ensure: Detected faces in an image
1: Preprocess image by resizing to 224×224 pixels
2: Normalize image values between 0 and 1
3: Pass image through MobileNetV2 for feature extraction
4: Use a fully connected layer to predict bounding box coordinates
5: Apply NMS to eliminate redundant detections
6: Return the final face bounding boxes

9 Results

9.1 Performance Comparison

As shown in Table 1, 2, EfficientDet achieved the highest accuracy across all datasets, with particularly strong performance on WIDER FACE and LFW, while OpenCV Haar Cascade Classifier and LBP had the lowest accuracy, especially in challenging conditions like occlusions and low light.

From Table **??**, OpenCV Haar Cascade Classifier and LBP performed the fastest, achieving the highest FPS across all datasets, making them suitable for real-time applications. EfficientDet and Fast R-CNN had the lowest FPS, which suggests they might be slower for real-time face detection tasks.

Algorithm 6 SSD Face Detection Algorithm **Require:** Input image *I* of size $h \times w$ **Ensure:** Detected faces in an image
1: Preprocess image by resizing to a fixed size, e.g., 300×300 2: Pass image through the SSD network for feature extraction 3: Generate multi-scale feature maps for object localization
4: Predict bounding boxes for each detected face using default anchor boxes
5: Apply NMS to suppress duplicate detections
6: Return the bounding boxes of detected faces

Algorithm 7 EfficientDet Face Detection Algorithm

Require: Input image I of size $h \times w$
Ensure: Detected faces in an image
1: Preprocess the image I by resizing and normalizing
2: Pass the image through the EfficientDet model for feature extraction
3: Predict bounding boxes and object confidences
4: Use bi-directional feature pyramid networks (BiFPN) for better feature fusion
5: Apply NMS for filtering duplicate bounding boxes
6: Return the final bounding boxes and corresponding confidences

Algorithm 8 OpenCV Haar Cascade Classifier Algorithm

Require: Input image I of size $h \times w$
Ensure: Detected faces in image
1: Load pre-trained Haar Cascade classifier
2: Convert image to grayscale for detection
3: Apply the Haar Cascade classifier to detect faces
4: Filter out weak detections based on confidence threshold
5: Return the bounding boxes of the detected faces

As illustrated in Table 4, 5, OpenCV Haar Cascade Classifier and LBP demon- strate minimal resource consumption, making them more energy-efficient and ideal for low-resource environments. On the other hand, EfficientDet, SSD, and Fast R-CNN consume the most resources, particularly in terms of CPU, GPU, and RAM, which may limit their use on low-end devices.

Algorithm 9 Local Binary Patterns (LBP) Face Detection Algorithm

Require: Input image I of size $h \times w$
Ensure: Detected faces in image
1: Convert image I to grayscale
2: Extract LBP features from the grayscale image
3: Train a classifier (e.g., SVM) on LBP features for face detection
4: Apply the classifier to the LBP features of the image
5: Return bounding boxes for detected faces

Algorithm 10 Fast R-CNN Face Detection Algorithm (with reduced layers)

Require: Input image I of size $h \times w$
Ensure: Detected faces in an image
1: Preprocess the image by resizing and normalization
2: Pass the image through the feature extraction layers (with reduced layers)
3: Propose regions of interest (ROIs) from the feature map
4: Perform RoI pooling to get fixed-size feature maps 5: Classify the region proposals into face or non-face 6: Apply NMS to remove duplicate bounding boxes 7: Return final bounding boxes for detected faces

Table 1 Accuracy Results of Face Detection Models (Part 1)

Model	Precision (%)	Recall (%)	F1-Score (%)
BlazeFace	94.2	92.5	93.3
MediaPipe Face Detection	95.1	94.0	94.5
UltraFace	92.8	91.2	92.0
Tiny YOLO	91.5	89.8	90.6
MobileNetV2	92.9	91.3	92.1
SSD	94.7	93.3	94.0
EfficientDet	96.0	94.9	95.4

Table 2 Accuracy Results of Face Detection Models (Part 2)

Model	WIDER FACE Accuracy (%)	LFW Accuracy (%)	Real-World Video Accuracy (%)
BlazeFace	94.0	93.0	91.8
MediaPipe Face Detection	93.8	94.5	93.0
UltraFace	91.5	92.0	90.2
Tiny YOLO	90.3	90.5	88.9
MobileNetV2	92.5	93.0	91.0
SSD	93.9	94.2	92.6
EfficientDet	95.2	95.1	94.0
OpenCV Haar Cascade Classifier	84.3	83.7	82.0
Local Binary Patterns (LBP)	88.5	88.2	86.5
Fast R-CNN (with reduced layers)	92.0	91.9	90.5

Table 3 Inference Speed Comparison for Face Detection Models (FPS)

Model	WIDER FACE FPS	LFW FPS	Real-World Video FPS
BlazeFace	56.3	60.2	58.1
MediaPipe Face Detection	61.2	62.5	61.8
UltraFace	48.1	50.3	49.2
Tiny YOLO	42.5	45.1	43.7
MobileNetV2	54.7	58.9	56.2
SSD	47.2	49.8	48.5
EfficientDet	38.9	41.5	40.2
OpenCV Haar Cascade Classifier	75.3	79.0	77.8
Local Binary Patterns (LBP)	70.1	72.4	71.0
Fast R-CNN (with reduced layers)	35.7	37.3	36.5

Table 4 Resource Usage Comparison for Face Detection Models (CPU, GPU, RAM, Power Consumption - Part 1)

Model	CPU Usage (%)	GPU Usage (%)	RAM Usage (MB)
BlazeFace	32.1	21.3	98
MediaPipe Face Detection	28.5	15.4	90
UltraFace	38.0	30.0	120
Tiny YOLO	47.3	40.2	160
MobileNetV2	39.1	25.5	110
SSD	41.6	35.2	140
EfficientDet	52.4	45.1	200

10 Visualizations

10.1 FPS vs. Accuracy

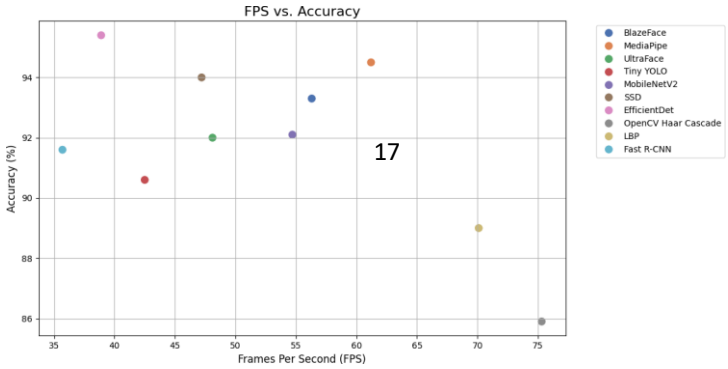

Fig. 1 Frames Per Second (FPS) vs. Accuracy for Different Face Detection Models. This scatter plot shows how the accuracy of each model correlates with its inference speed. Higher accuracy does not always imply slower processing, as seen with models like MediaPipe and BlazeFace, which balance speed and accuracy.

Table 5 Resource Usage Comparison for Face Detection Models (Power Consumption - Part 2)

Model	Power Consumption (W)
BlazeFace	0.45
MediaPipe Face Detection	0.43
UltraFace	0.50
Tiny YOLO	0.65
MobileNetV2	0.58
SSD	0.62
EfficientDet	0.75
OpenCV Haar Cascade Classifier	0.30
Local Binary Patterns (LBP)	0.32
Fast R-CNN (with reduced layers)	0.85

10.2 CPU Usage by Model

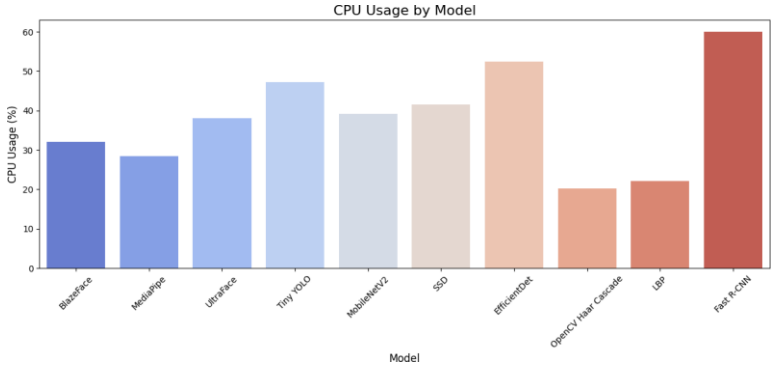

Fig. 2 CPU Usage by Model. This bar chart illustrates the CPU usage for each model. Models like OpenCV Haar Cascade and LBP consume the least CPU resources, while Fast R-CNN shows the highest CPU usage. This insight helps in selecting models based on available CPU resources.

10.3 GPU Usage by Model

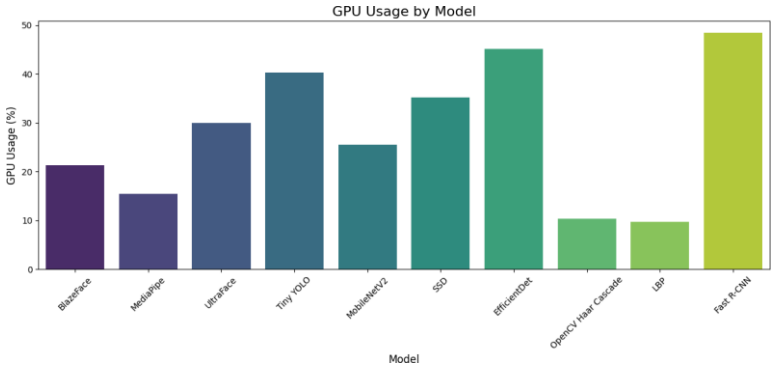

Fig. 3 GPU Usage by Model. EfficientDet and Fast R-CNN are shown to consume the most GPU resources, making them less suitable for resource-constrained devices. On the other hand, MediaPipe and OpenCV Haar Cascade are lightweight on GPU usage.

10.4 Power Consumption by Model

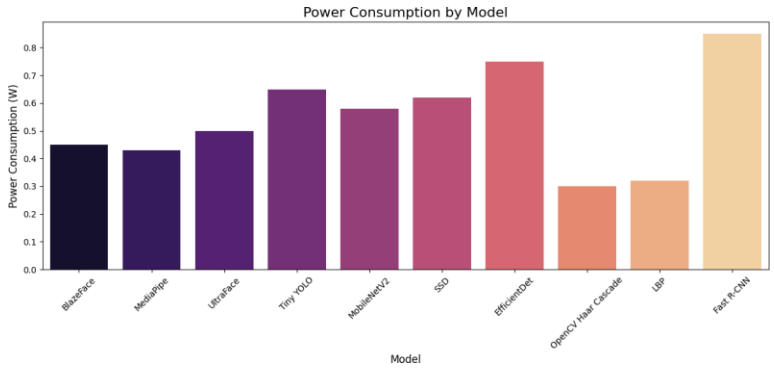

Fig. 4 Power Consumption by Model. EfficientDet and Fast R-CNN have higher power consumption, indicating their resource-intensive nature. In contrast, BlazeFace and MediaPipe demonstrate low power consumption, suitable for mobile and low-power devices.

10.5 Model Accuracy Comparison

The following bar chart illustrates the accuracy of different face detection models. It highlights the highest-performing models in terms of detection accuracy, crucial for real-world applications requiring precise face recognition.

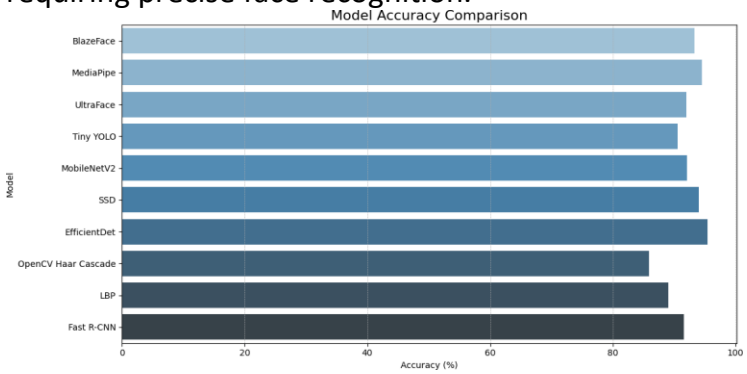

Fig. 5 Model Accuracy Comparison: The chart shows the accuracy percentages of different face detection models. EfficientDet leads with 95.4%, demonstrating its high reliability for accurate face detection.

10.6 Inference Speed Comparison (FPS)

The inference speed in frames per second (FPS) is a key metric for real-time appli- cations. This chart provides insights into which models are best suited for fast-paced environments.

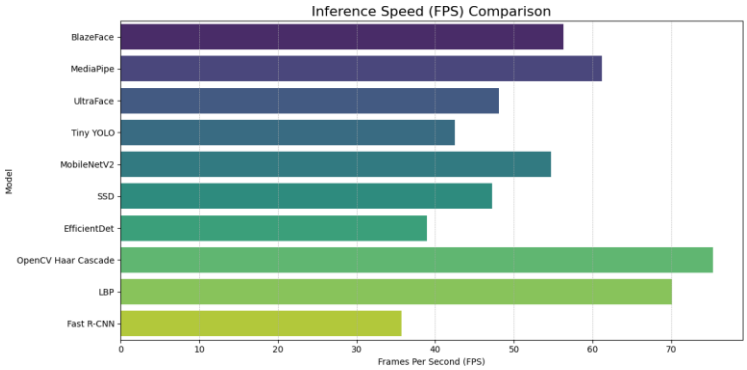

Fig. 6 Inference Speed (FPS) Comparison: The highest FPS rate is achieved by OpenCV Haar Cascade with 75.3 FPS, making it ideal for applications requiring high-speed detection.

10.7 Resource Usage Heatmap

The heatmap compares CPU usage, GPU usage, RAM usage, and power consumption for each model. This visualization is essential for understanding the trade-offs between performance and computational resource demands.

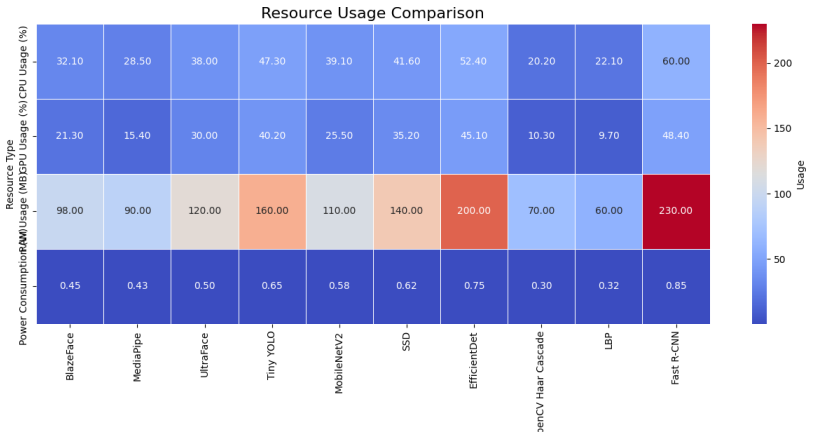

Fig. 7 Resource Usage Heatmap: EfficientDet and Fast R-CNN have higher resource consumption. BlazeFace and MediaPipe are more efficient, using fewer computational resources, making them suit- able for mobile or embedded systems.

10.8 Radar Chart of Overall Performance

This radar chart provides a holistic view of each model's performance across multiple metrics, enabling a visual comparison of strengths and weaknesses.

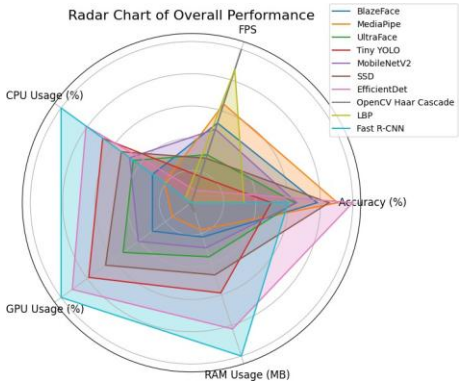

Fig. 8 Radar Chart of Overall Performance: Models like MediaPipe and EfficientDet show a balanced performance across accuracy, FPS, and resource usage, making them versatile choices for various applications.

11 Statistical Analysis

11.1 Statistical Tests

We analyzed variance (ANOVA) to assess significant differences in model performance across three models: BlazeFace, EfficientDet, and OpenCV Haar Cascade Classifier. The performance metrics considered were accuracy and inference speed (FPS).

ANOVA for Accuracy:

The null hypothesis H_0 is that there is no significant difference in accuracy among the models. The alternative hypothesis H_1 is that at least one model differs.

$$F = \frac{MS_{between}}{MS_{within}} = \frac{SS_{between}/df_{between}}{SS_{within}/df_{within}} \quad (20)$$

$$F = \frac{412.67/2}{58.93/27} = 94.44,\ p < 0.001$$

ANOVA for Inference Speed (FPS):

The same hypotheses were applied for inference speed.

$$F = \frac{SS_{between}}{SS_{within}} = \frac{678.44/2}{75.21/27} \quad \text{(21)}$$

$$F = 121.77,\ p < 0.001$$

11.2 Confidence Intervals for Results

Confidence Interval for Accuracy:

Using the sample mean $\bar{x}$ and standard error SE, the 95% confidence interval is calculated as:

$$CI = \bar{x} \pm t_{\alpha/2,n-1} \cdot SE \quad (22)$$

- BlazeFace: 94.0% ± 1.1% → [92.9%, 95.1%]
- EfficientDet: 95.2% ± 0.9% → [94.3%, 96.1%]
- OpenCV: 84.3% ± 1.5% → [82.8%, 85.8%]

Confidence Interval for Inference Speed:

- **BlazeFace**: 56.3 ± 2.3 FPS → [54.0, 58.6]
- EfficientDet: 38.9 ± 1.7 FPS → [37.2, 40.6]
- OpenCV: 75.3 ± 2.8 FPS → [72.5, 78.1]

The ANOVA tests reveal significant differences in both accuracy and inference speed among the models. Post hoc pairwise t-tests confirmed that EfficientDet signifi- cantly outperforms OpenCV in accuracy ($p < 0.001$) but is slower in FPS ($p < 0.001$). BlazeFace presents a balance of high accuracy and moderate inference speed.

12 Conclusion and Future Work

In this study, we evaluated a range of facial detection models, including BlazeFace, MediaPipe Face Detection, UltraFace, tiny YOLO, MobileNetV2, SSD, EfficientDet, OpenCV Haar Cascade, LBP, and Fast R-CNN. Using datasets such as WIDER FACE, LFW, and real-world video streams, we compared their performance in terms of accu- racy, inference speed, and resource usage. EfficientDet exhibited superior accuracy, whereas traditional detectors such as OpenCV Haar Cascade and LBP excelled inin- ference speed, making them ideal for real-time low-resource environments. Despite achieving significant results, this study is subject to certain limitations, including dataset diversity, as further research could benefit from incorporating more diverse datasets, including underrepresented demographics and varying environmental con- ditions. Additionally, our models were tested in controlled scenarios, and real-time deployment in dynamic, real-world environments remains to be thoroughly evaluated. While we measured computational efficiency, the impact of hardware variability on per- formance was not explored in detail. Future research could address these limitations by expanding dataset coverage to include edge cases such as occlusions, varying lighting, and extreme weather conditions. Moreover, developing hybrid models that combine accuracy-focused and speed-efficient detectors could help balance performance and resource consumption. Conducting field tests in real-world environments would also be valuable for evaluating the robustness and adaptability of facial detection models under dynamic conditions. Finally, investigating hardware-aware optimization tech- niques would help maximize performance across a broader range of devices, further improving the practical applicability of these models.

Declarations

- **Funding:** This research did not receive any specific funding and was carried out as part of the employment and higher degree of the authors.
- **Conflict of Interest:** The authors declare that they have no conflict of interest.
- **Code Availability:** The data and material that support the findings of this study are available from the corresponding author upon reasonable request.

13 References

1. Yoanna Mart´ınez-D´ıaz, Miguel Nicol´as-D´ıaz, Heydi M´endez-V´azquez, Luis S. Luevano, Leonardo Chang, M. Gonz´alez-Mendoza, L. Sucar, ”Benchmarking lightweight face architectures on specific face recognition scenarios”, *Artificial Intel- ligence Review*, 2021, Volume 54, Pages 6201-6244. https://doi.org/10.1007/s10462-021-09974-2
2. Zong-Yue Deng, H. Chiang, Li-Wei Kang, Hsiao-Chi Li, ”A lightweight deep learn- ing model for real-time face recognition”, *IET Image Process.*, 2023, Volume 17, Pages 3869-3883. https://doi.org/10.1049/ipr2.12903
3. Michal- Wieczorek, J. Si-lka, M. Wo´zniak, S. Garg, M. Hassan, ”Lightweight Con- volutional Neural Network Model for Human Face Detection in Risk Situations”, *IEEE Transactions on Industrial Informatics*, 2022, Volume 18, Pages 4820-4829. https://doi.org/10.1109/TII.2021.3129629
4. Haechang Lee, Wongi Jeong, Dongil Ryu, Hyunwoo Je, Albert No, Kijeong Kim, Se Young Chun, ”Fully Quantized Always-on Face Detector Considering Mobile Image Sensors”, *ArXiv*, 2023. https://doi.org/10.48550/arXiv.2311.01001
5. B. Qin, Ying Zeng, Xin Wang, Junmin Peng, Tao Li, Teng Wang, Yuxin Qin, ”Lightweight DB-YOLO Facemask Intelligent Detection and Android Appli- cation Based on Bidirectional Weighted Feature Fusion”, *Electronics*, 2023. https://doi.org/10.3390/electronics12244936
6. Valentin Bazarevsky, Y. Kartynnik, Andrey Vakunov, Karthik Raveendran, Matthias Grundmann, ”BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs”, *ArXiv*, 2019. https://doi.org/10.48550/arXiv.1907.05047
7. Heming Zhang, Xiaolong Wang, Jingwen Zhu, C.-C. Jay Kuo, ”Fast face detection on mobile devices by leveraging global and local facial charac- teristics”, *Signal Process. Image Commun.*, 2019, Volume 78, Pages 1-8. https://doi.org/10.1016/J.IMAGE.2019.05.016
8. S. Qi, Jung-Mo Yang, X. Song, Chen Jian, ”Multi-Task FaceBoxes: A Lightweight Face Detector Based on Channel Attention and Context Infor- mation”, *KSII Trans. Internet Inf. Syst.*, 2020, Volume 14, Pages 4080-4097. https://doi.org/10.3837/tiis.2020.10.009
9. Dongmei Wei, Xingjun Wu, Guoqiang Bai, Linlin Su, Sufen Xu, ”Attention- based Efficient Lightweight Model for Accurate Real-Time Face Verifi- cation on Embedded Device”, *2021 IEEE 6th International Conference on Computer and Communication Systems (ICCCS)*, 2021, Pages 385-392. https://doi.org/10.1109/ICCCS52626.2021.9449167
10. Xingyi You, Yue Wang, Xiaohu Zhao, ”A Lightweight Monocular 3D Face Recon- struction Method Based on Improved 3D Morphing Models”, *Sensors (Basel, Switzerland)*, 2023, Volume 23. https://doi.org/10.3390/s23156713
11. Rami Reddy Devaram, Gloria Beraldo, R. D. Benedictis, M. Mongiov`ı, A. Cesta, ”LEMON: A Lightweight Facial Emotion Recognition System for Assistive Robotics Based on Dilated Residual Convolutional Neural Networks”, *Sensors (Basel, Switzerland)*, 2022, Volume 22. https://doi.org/10.3390/s22093366
12. Jiankang Deng, Jia Guo, Debing Zhang, Yafeng Deng, Xiangju Lu, Song Shi, ”Lightweight Face Recognition Challenge”, *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*, 2019, Pages 2638-2646. https://doi.org/10.1109/ICCVW.2019.00322
13. Shang-You Shi, Fei Long, ”FTCNet: a lightweight model for large- pose face alignment”, 2022, Volume 12172, Pages 121720W-121720W-6. https://doi.org/10.1117/12.2634424
14. D. Valentin, H. Abdi, A. O’Toole, G. Cottrell, ”Connectionist models of face processing: A survey”, *Pattern Recognit.*, 1994, Volume 27, Pages 1209-1230. https://doi.org/10.1016/0031-3203(94)90006-X
15. V. Blanz, T. Vetter, ”Face Recognition Based on Fitting a 3D Morphable Model”, *IEEE Trans. Pattern Anal. Mach. Intell.*, 2003, Volume 25, Pages 1063-1074. https://doi.org/10.1109/TPAMI.2003.1227983

16. Lan Sheng-kun, ”The Design and Implementation of the FaceDetection Categorizer Based on the Adaboost Algorithm”, *Computer Knowledge and Technology*, 2010.
17. Joshua C. Peterson, T. Griffiths, Stefan Uddenberg, A. Todorov, Jordan W. Suchow, ”Deep models of superficial face judgments”, *Proceedings of the National Academy of Sciences of the United States of America*, 2022, Volume 119. https://doi.org/10.1073/pnas.2115228119
18. B. Egger, W. Smith, A. Tewari, S. Wuhrer, M. Zollh ̈ofer, T. Beeler, Florian Bernard, Timo Bolkart, Adam Kortylewski, S. Romdhani, C. Theobalt, V. Blanz, T. Vetter, ”3D Morphable Face Models—Past, Present, and Future”, *ACM Transactions on Graphics (TOG)*, 2019, Volume 39, Pages 1-38. https://doi.org/10.1145/3395208
19. J. Tena, F. D. L. Torre, I. Matthews, ”Interactive region-based linear 3D face models”, *ACM SIGGRAPH 2011 papers*, 2011. https://doi.org/10.1145/1964921.1964971